Introduction to Intensive Programming in C++

Lecture 4: Greedy algorithms, codes, and trie

Elias TSIGARIDAS

École Polytechnique

Contents

Greedy algorithms

General strategy
It is a technique, not an algorithm.

Pick the most obvious solution!

proof strategy !!!

- Assume that there is an optimal solution that is different from the greedy solution.
- Find the *first* difference between the two solutions.
- *Exhange argument*: Argue that we can exchange the optimal choice for the greedy choice without making the solution worse (although the exchange might not make it better).

Greedy algorithms

General strategy
It is a technique, not an algorithm.

Pick the most obvious solution!

proof strategy !!! ← this is the hard part

- Assume that there is an optimal solution that is different from the greedy solution.
- Find the *first* difference between the two solutions.
- *Exhange argument*: Argue that we can exchange the optimal choice for the greedy choice without making the solution worse (although the exchange might not make it better).

- Greedy cannot handle problems with (many) local mins
- Exchange argument (essentially) proves that there is no local min

Other greedy algorithms

- Coin exchange
- Optimal caching
- *Minimum spanning tree (Kruskal, Prim)*
- *(single pair) Shortest path*

Optimal chaching

Problem: n files of length $L[1..n]$ are stored sequentially on a tape T

cost of accessing the k-th file is $\text{cost}(k) = \sum_{i=1}^{k} L[i]$

Store the files in a way to minimize the expected cost

Optimal chaching

Problem: n files of length $L[1..n]$ are stored sequentially on a tape T

cost of accessing the k-th file is $\text{cost}(k) = \sum_{i=1}^{k} L[i]$

Store the files in a way to minimize the expected cost

If every file is equally likely, then $\mathbb{E}[\text{cost}] = \sum_{k=1}^{n} \frac{\text{cost}(k)}{n} = \frac{1}{n} \sum_{k=1}^{n} \sum_{i=1}^{k} L[i]$

BUT different orders give different expected costs

Consider a permutation π then

$$\mathbb{E}[\text{cost}(\pi)] = \frac{1}{n} \sum_{k=1}^{n} \sum_{i=1}^{k} L[\pi(i)]$$

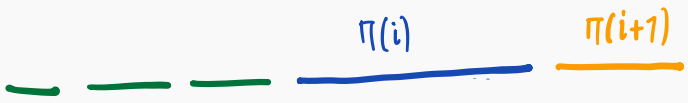
SORT the files in increasing length

GREEDY: put the cheapest to access file first

WHY ?

Optimal chaching

LEMMA: $\min \mathbb{E}[\text{cost}(\pi)]$ if $L[\pi(i)] \leq L[\pi(i+1)] \quad \forall\, i$

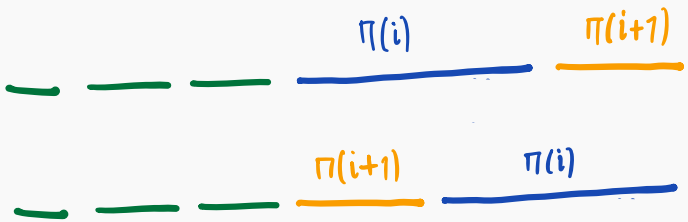


Optimal chaching

LEMMA: $\min \mathbb{E}[\text{cost}(\pi)]$ if $L[\pi(i)] \leq L[\pi(i+1)] \quad \forall\, i$

proof:

Assume $\exists\, i$ s.t. $L[\pi(i)] > L[\pi(i+1)]$



then if we swap $\pi(i)$ and $\pi(i+1)$

- the cost of $\pi(i)$ increases (+) by $L[\pi(i+1)]$
- the cost of $\pi(i+1)$ decreases (-) by $L[\pi(i)]$

the difference is $L[\pi(i+1)] - L[\pi(i)] \leq 0$

so we reduce the cost, if we swap pairs that are not in order.

Optimal chaching

Variant : n files , lengths $L[1..n]$, frequences $F[1..n]$

$$Fcost(\pi) = \sum_{k=1}^{n} \left(F[\pi(k)] \cdot cost(\pi(k)) \right) = \sum_{k=1}^{n} \left(F[\pi(k)] \cdot \sum_{i=1}^{k} L[\pi(i)] \right)$$

GREEDY : But how to sort now ?

Optimal chaching

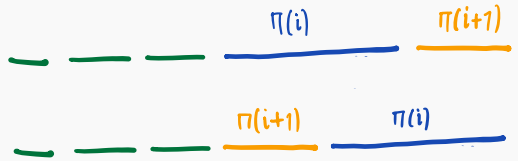
Variant : n files, lengths $L[1..n]$, frequences $F[1..n]$

$$Fcost(\pi) = \sum_{k=1}^{n} \left(F[\pi(k)] \cdot cost(\pi(k)) \right) = \sum_{k=1}^{n} \left(F[\pi(k)] \cdot \sum_{i=1}^{k} L[\pi(i)] \right)$$

GREEDY : But how to sort now ?

LEMMA : $\min Fcost(\pi)$ when $\frac{L[\pi(i)]}{F[\pi(i)]} \leq \frac{L[\pi(i+1)]}{F[\pi(i+1)]} \quad \forall i$

Assume $\exists$ i s.t. $\frac{L[\pi(i)]}{F[\pi(i)]} \geq \frac{L[\pi(i+1)]}{F[\pi(i+1)]}$ then swap them



- for $\pi(i)$ the cost increases (+) by $F[\pi(i)] \cdot L[\pi(i+1)]$
- for $\pi(i+1)$ the cost decreases (−) by $F[\pi(i+1)] \cdot L[\pi(i)]$

So in total $F[\pi(i)] \cdot L[\pi(i+1)] - F[\pi(i+1)] L[\pi(i)] \leq 0$ So we reduce the cost if we swap

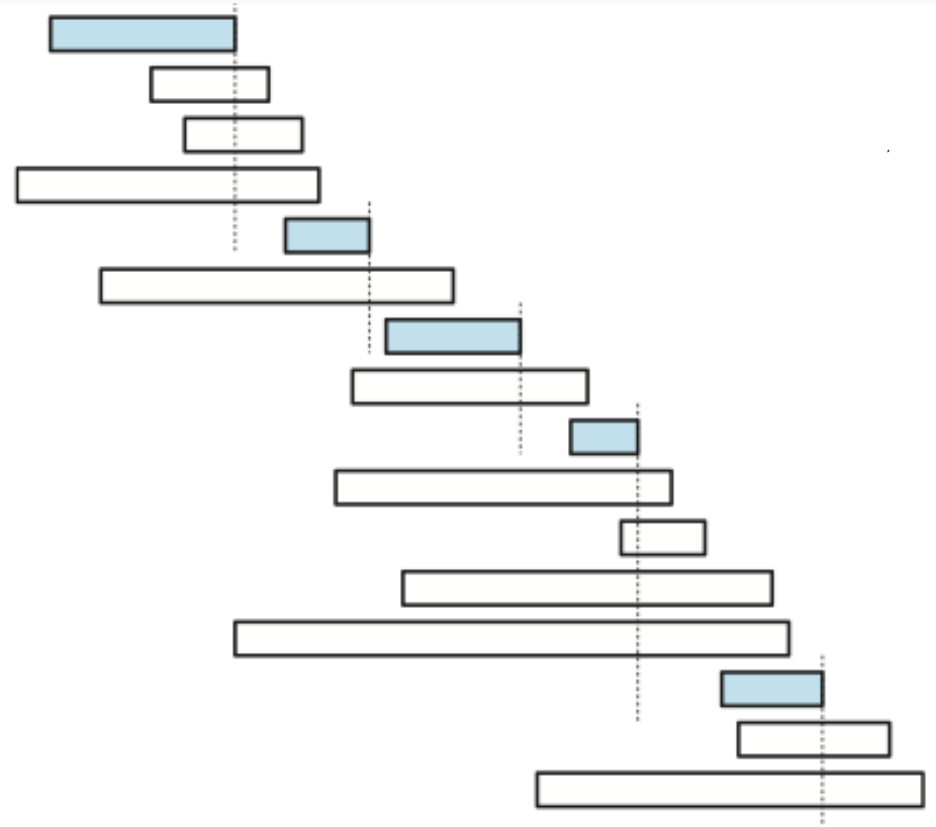
Class scheduling

n classes

starting times $S[1..n]$

finishing times $F[1..n]$

Maximize the number of classes that you attend



```
GREEDYSCHEDULE(S[1..n], F[1..n]):
  sort F and permute S to match
  count ← 1
  X[count] ← 1
  for i ← 2 to n
      if S[i] > F[X[count]]
          count ← count + 1
          X[count] ← i
  return X[1..count]
```

Class scheduling

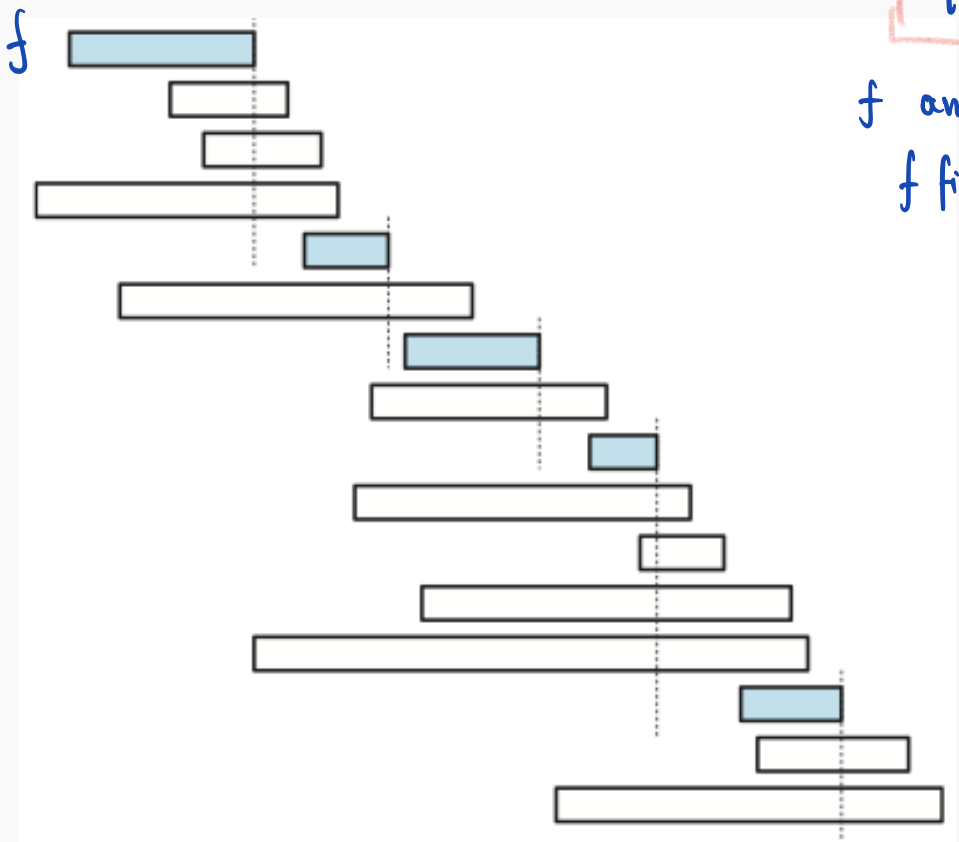
n classes

starting times $S[1..n]$

finishing times $F[1..n]$

Maximize the number of classes that you attend

At least one maximal conflict-free schedule includes the first finishing class



f and $X = \{g_1, g_2, \ldots, g_k\}$

f finishes before g_1 so $(X \setminus \{g_1\}) \cup \{f\}$ is conflict-free and optimal

```
GREEDYSCHEDULE(S[1..n], F[1..n]):
  sort F and permute S to match
  count ← 1
  X[count] ← 1
  for i ← 2 to n
      if S[i] > F[X[count]]
          count ← count + 1
          X[count] ← i
  return X[1..count]
```

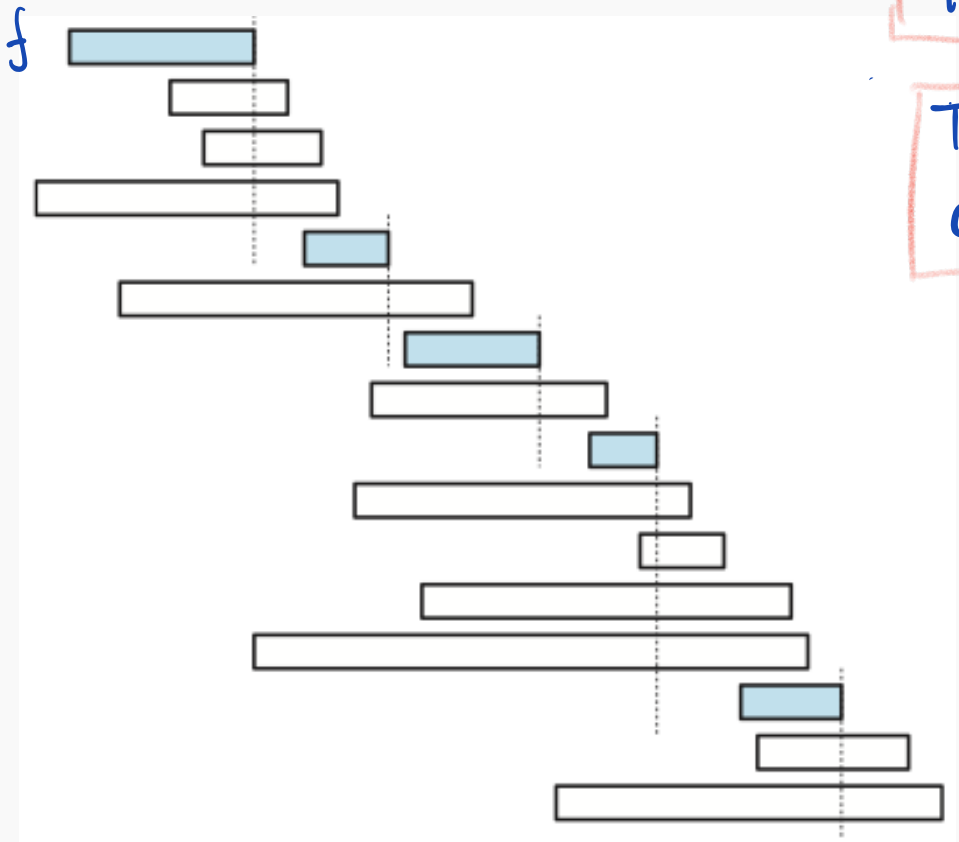
Class scheduling

n classes

starting times $S[1..n]$

finishing times $F[1..n]$

Maximize the number of classes that you attend



At least one maximal conflict-free schedule includes the first finishing class

The greedy produces an optimal conflict-free schedule

induction by generalizing

```
GREEDYSCHEDULE(S[1..n], F[1..n]):
  sort F and permute S to match
  count ← 1
  X[count] ← 1
  for i ← 2 to n
      if S[i] > F[X[count]]
          count ← count + 1
          X[count] ← i
  return X[1..count]
```

Complexity: sorting + selecting $O(n \lg n) + O(n) = O(n \lg n)$

Greedy children

There are cookies, with sizes s_j

There are children with greed factor g_i.

If $s_j \geq g_i$ we can give the cookie j to child i and the child is happy.

Maximize the number of happy children.

```
if i < s.size() and j < g.size()
   if s[i] >= g[j] {
     i++;
     j++;
     r++;
   } else {
     i++;
   }
}
```

Sort s and g (in which order?)

Minimum number of platforms

The input is the arrival and departure time of n trains in a station.

Find the mimimum number of platforms needed to accomodate all trains in the station.

- Sort the two arrays.
- Increase or decrease a counter.
- If an arrival and a departure happen at the same time, give priority to the departure.

Construct the maximum number

Given two arrays of lenght m and n representing two numbers.

Construct a maximum number with $k \leq m + n$ digits.

We should preserve the relative order of the digits of each array.

$$\mathtt{A} = [6, 7]$$

$$\mathtt{B} = [6, 0, 4]$$

$$k = 5$$

$$\mathtt{R} = [6, 7, 6, 0, 4]$$

Smallest range covering elements from k lists

You have k lists of sorted integers in non-decreasing order.

Find the smallest range that includes at least one number from each of the k lists.

$$[a, b] < [c, d] \text{ if } \begin{cases} b - a < d - c \\ \text{or} \\ b - a = d - c \text{ and } a < c \end{cases}$$

$k=3$
$n=6$

$[4, 7, 9, 12, 15]$
$[0, 8, 10, 14, 20]$
$[6, 12, 16, 30, 50]$

result $[6, 8]$

'Sliding Window'

Binary codes

constant length

char	code
A	00
B	01
C	10
D	11

enumeration

not valid

char	code
A	10
B	11
C	1011
D	111

optimal

char	code
A	0
B	110
C	10
D	111

INPUT

00 00 01 00 10 11 00 10 00

A A B A C D A C A

length of the message = 18

10 11

A B

or C

0 0 110 0 10 111 0 10 0

A A B A C D A C A

length of the message = 15

Huffman codes

Prefix-free code

No codeword is a prefix of any other codeword

Huffman codes [~ 1951]

- Let f_i be the number of occurences of letter i.
- Total number of symbols needed

$$\sum_i f_i \, \text{length}(i)$$

- Huffman algorithms finds the optimal prefix-free code.

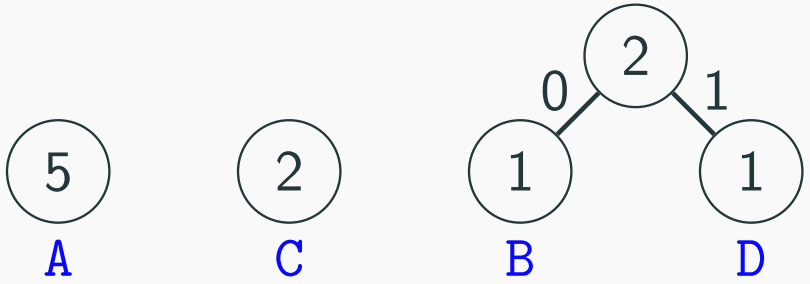
Variable length code

What does it mean?
Is there a lower bound?

extends binary trees to minimum weighted path length from given weights

Merge the two least frequent letters and recurse

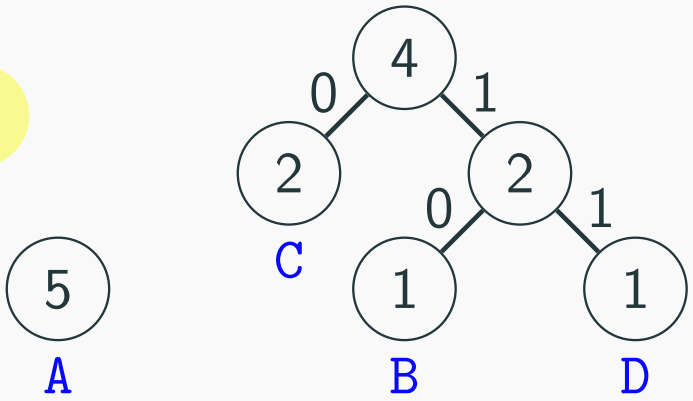
binary code and compression



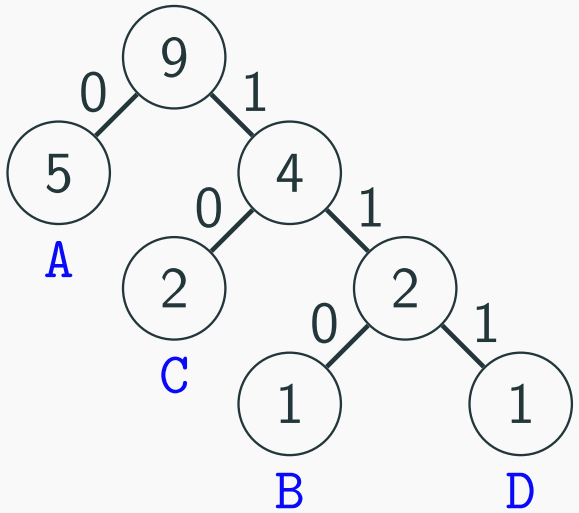
char	code
A	0
B	110
C	10
D	111

not unique

why?



AABACDACA



construction : priority queue → binary heap

Complexity : O(n) queries, each costs O(lgn)

O(nlgn)

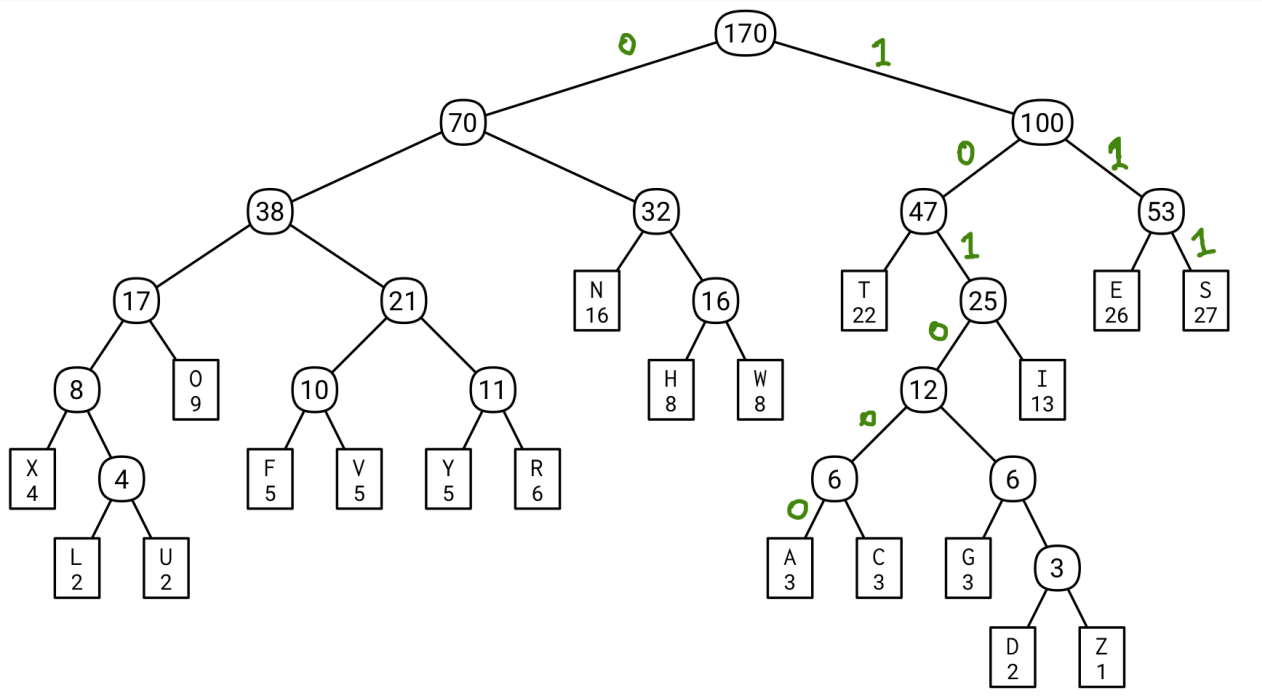
Demo Huffman tree

Hauffman codes : another example

This sentence contains three a's, three c's, two d's, twenty-six e's, five f's, three g's, eight h's, thirteen i's, two l's, sixteen n's, nine o's, six r's, twenty-seven s's, twenty-two t's, two u's, five v's, eight w's, four x's, five y's, and only one z.

```
THISSENTENCECONTAINSTHREEASTHREECSTWODSTWENTYSIXESFIVEFST
HREEGSEIGHTHSTHIRTEENISTWOLSSIXTEENNSNINEOSSIXRSTWENTYSEV
ENSSTWENTYTWOTSTWOUSFIVEVSEIGHTWSFOURXSFIVEYSANDONLYONEZ
```

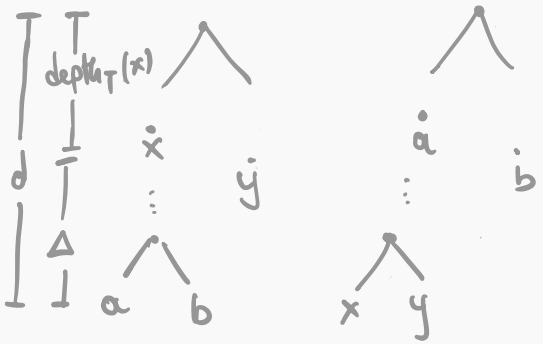
A	C	D	E	F	G	H	I	L	N	O	R	S	T	U	V	W	X	Y	Z
3	3	2	26	5	3	8	13	2	16	9	6	27	22	2	5	8	4	5	1



Huffman Codes — some details

LEMMA: Let x, y be two least frequent letters (break ties arbitrarily)
$\exists$ optimal code tree where x, y are leafs at the largest possible depth.

optimal T | new tree T'



$\Delta = d - \text{depth}_T(x)$

after swap:

- the depth of x increases by Δ (+) (cost: $+\Delta f[x]$)
- the depth of a decreases by Δ (−) (cost: $-\Delta f[a]$)

$$\text{cost}(T') = \text{cost}(T) + \Delta f[x] - \Delta \cdot f[a] = \text{cost}(T) + \Delta\underbrace{(f[x] - f[a])}_{\text{negative}}$$

But $f[x] \le f[a]$ so $\text{cost}(T') \le \text{cost}(T)$

However T optimal, thus $\text{cost}(T) \le \text{cost}(T')$

$\Big\}\ \text{cost}(T) = \text{cost}(T')$

Thm Every Huffman code is optimal prefix-free binary code

Information Theory

$A = \{a_1, \dots, a_n\}$ random variable $X \in A$

probabilities p_1 p_n $\sum p_i = 1$

Entropy $H(X) = -\sum_{x \in A} p(x) \lg p(x)$

Proposition: $0 \leq H(X) \leq \lg |A|$

for a r.v. X we need $H(X)$ bits to describe it on the average

LZW Lempel - Ziv - Welch

fixed length 12 bits encoding

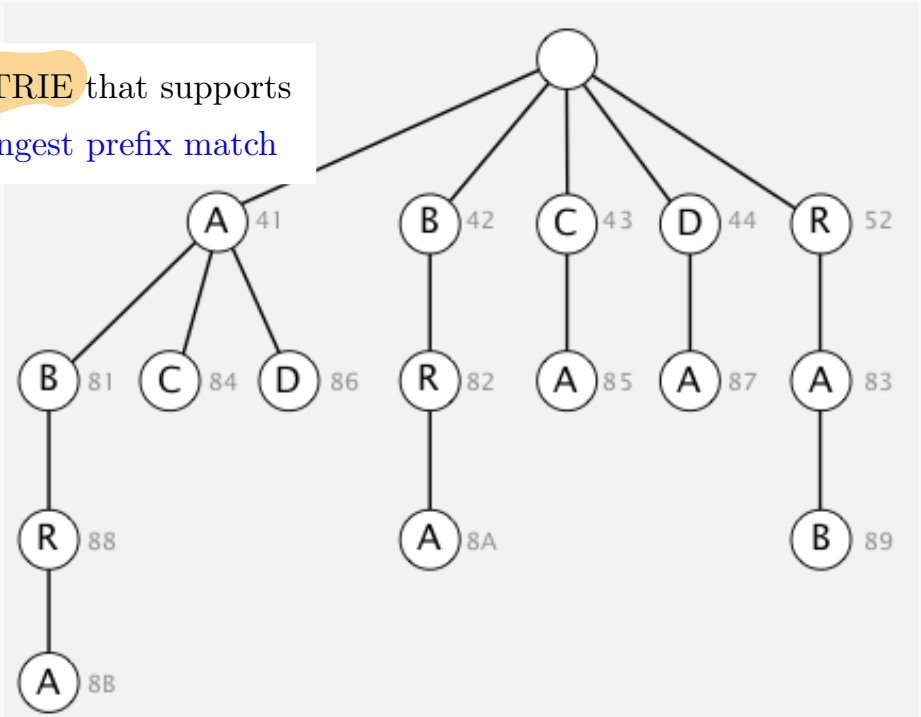
0-255
8 bits for one letter
4 bits for sequences
256-4095

input	A		B		R	A	C	A	D	A	B	R	A	B	R	A	B	R
matches	A		B		R	A	C	A	D	A B		R A		B R		A B R		
value	41		42		52	41	43	41	44	81		83		82		88		

key	value
⋮	⋮
A	41
B	42
C	43
D	44
⋮	⋮

key	value
AB	81
BR	82
RA	83
AC	84
CA	85
AD	86

a TRIE that supports longest prefix match

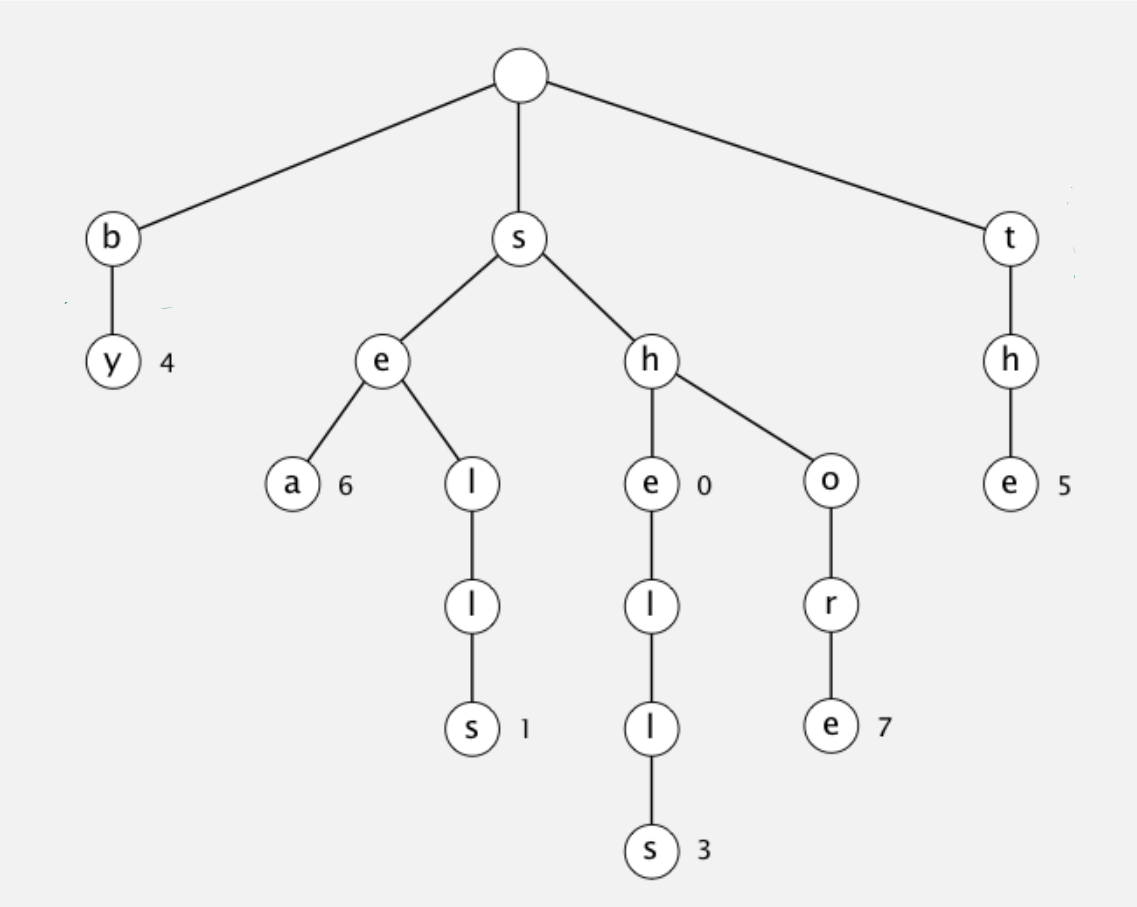


it can encode combinations

Contents

Trie: a prefix tree from reTRIEval

- You prononouce it like "tree", not like "try" (most of the times).
- Every node has (at most) r children
  (i.e., the number of symbols in the alphabet)

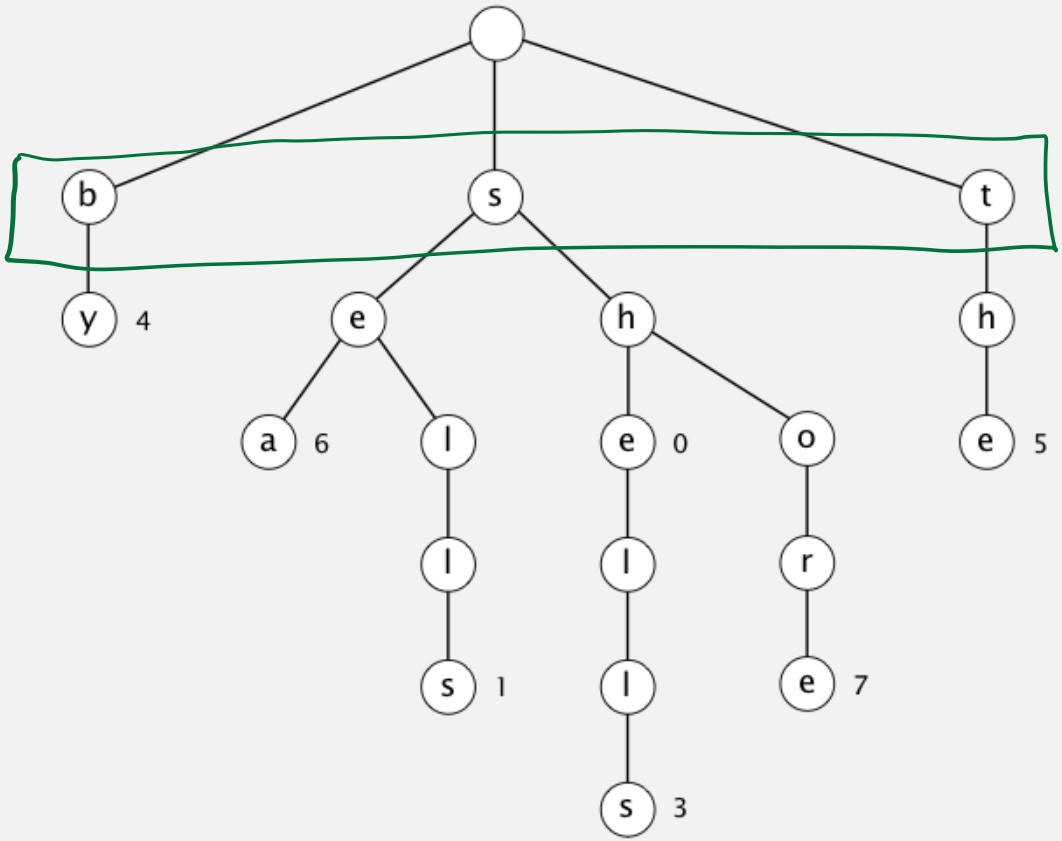


TQ

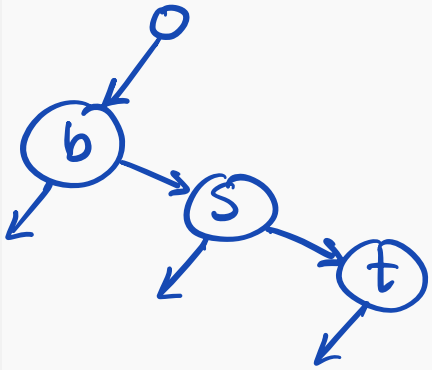
Can you convert any tree to a binary tree?

Trie: a prefix tree from reTRIEval

- You prononouce it like "tree", not like "try" (most of the times).
- Every node has (at most) r children
  (i.e., the number of symbols in the alphabet)



r-way tree



All the trees are binary trees

TQ

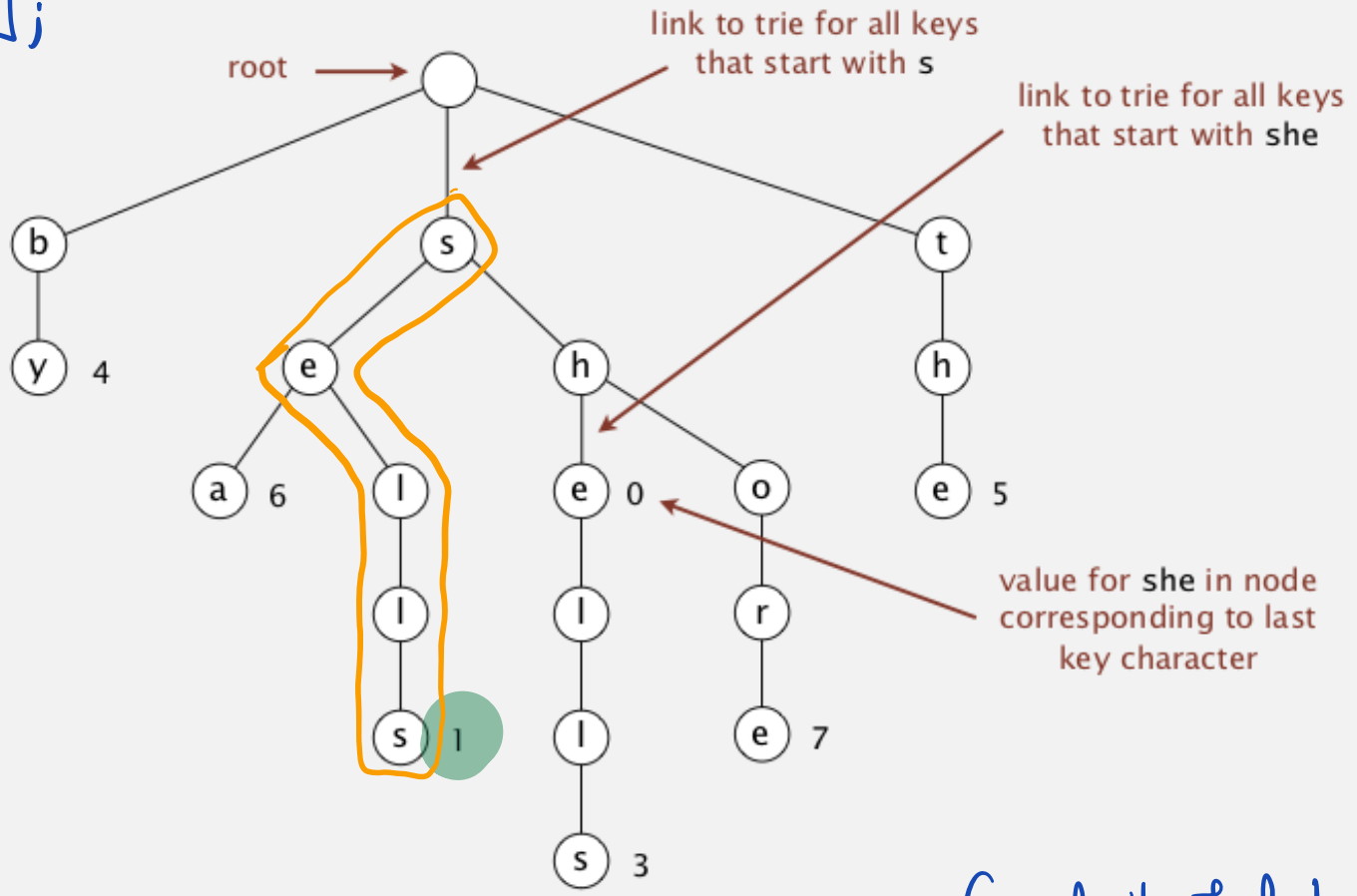
Can you convert any tree to a binary tree?

Complexity of constructing it?

Trie: an example

Implementation

```
struct Trie {
    Trie* children[r];
}
```



key	value
by	4
sea	6
sells	1
she	0
shells	3
shore	7
the	5

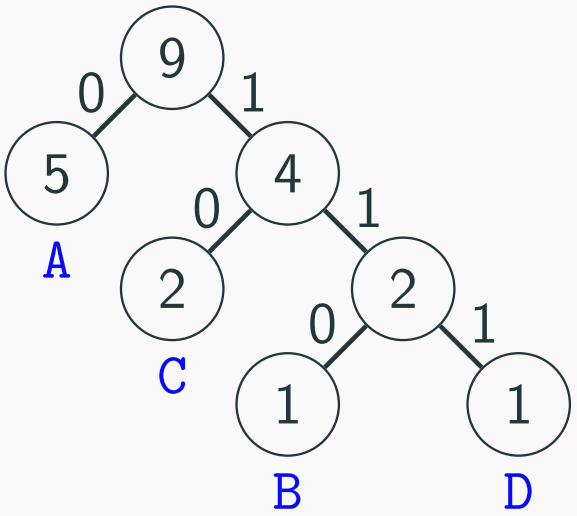
Complexity of lookup: O(length of word)

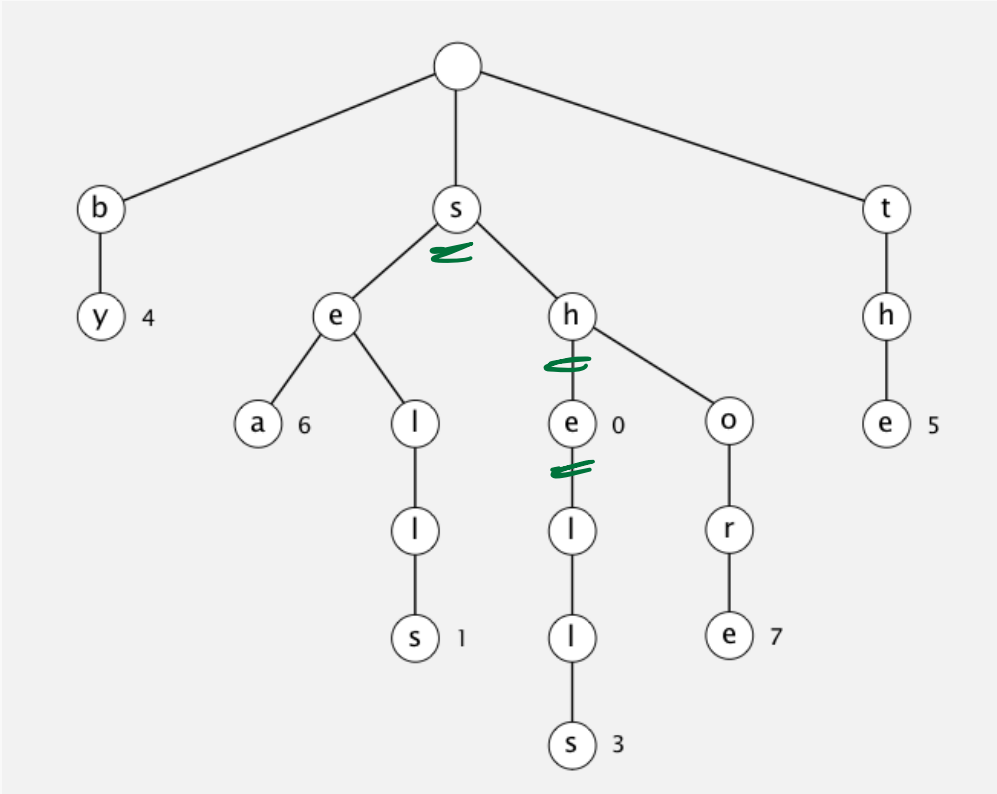
Lookup is independent of the number of strings in the trie!

Fast prefix search, but needs a lot of memory.

Can you tell the difference?

eg. she





Some questions

TQ

Find the longest word in a dictionary.

DFS
in trie data structure

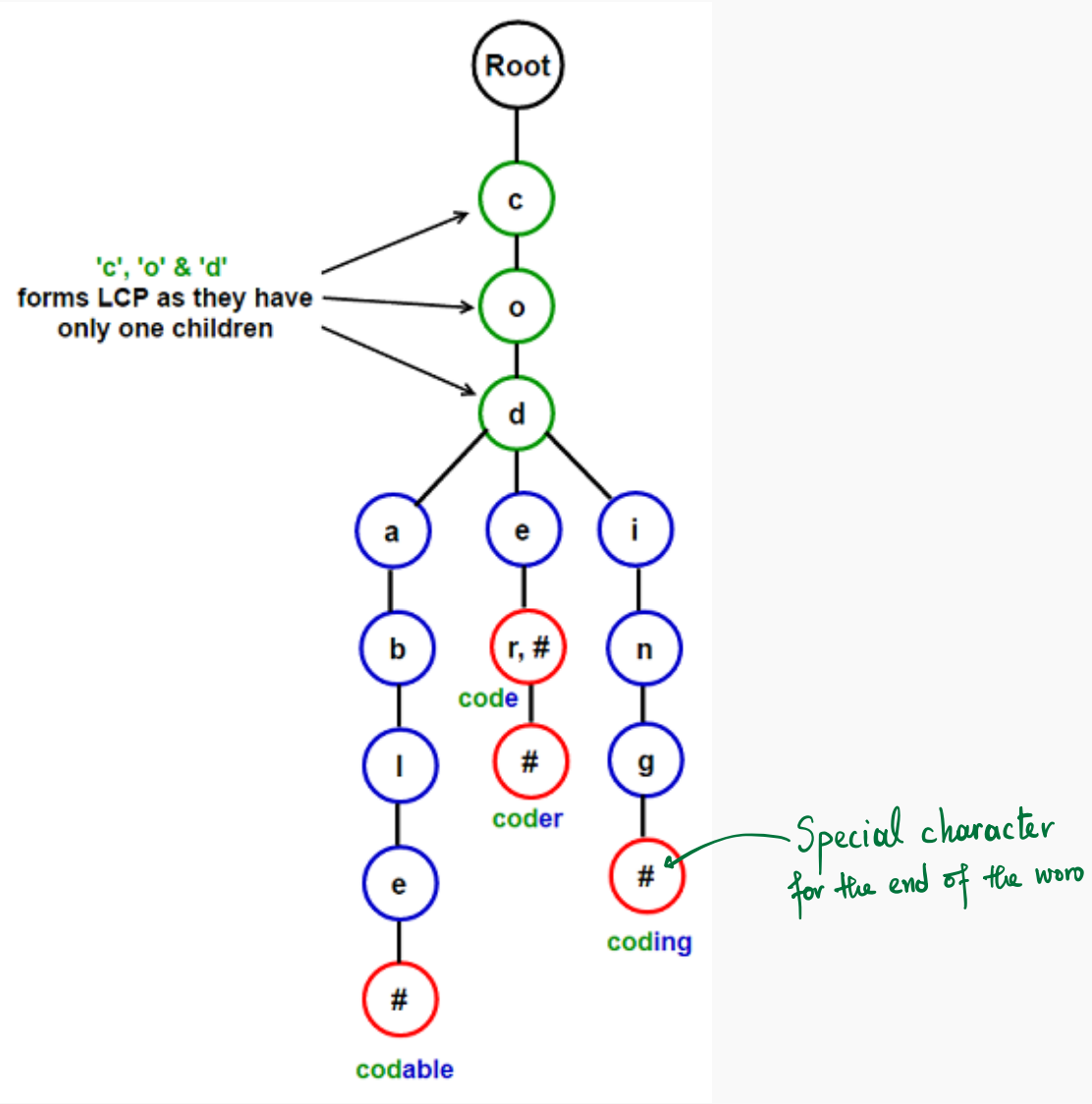
TQ

Write a web(?) application which provides instant search over the properties of some *some database*. The app does not wait for the user to press a submit button, but it dynamically updates the search results as input is typed.

TQ: Longest common prefix

LZW Compression

keys = [codable, code, coder, coding]



Applications of Trie

- dictionary lookup
- prefix searches
- auto complete feature in text editors
- command completion in IDE, editors, etc
- phone number/contacts searching
- IP lookups
- matching sentences (if a trie is based on a sequence of words, like Gmail autocompletion)
- finding positions of words in a text